

MATERIAŁ REFERENCYJNY

# Databricks Fundamental

Kompendium do jednodniowego szkolenia

---

# Spis treści

---

|                    |                             |    |
|--------------------|-----------------------------|----|
| <b>MODUŁ 01</b>    | Platforma i workspace       | 3  |
| <b>MODUŁ 02</b>    | Architektura Spark          | 6  |
| <b>MODUŁ 03</b>    | ELT i ingestia              | 7  |
| <b>MODUŁ 04</b>    | Delta Lake                  | 10 |
| <b>MODUŁ 05</b>    | Orkestracja w Lakeflow Jobs | 12 |
| <b>MODUŁ 06</b>    | Unity Catalog               | 14 |
| <b>MODUŁ 07</b>    | Dashboardy AI/BI i Genie    | 16 |
| <b>ZAŁĄCZNIK A</b> | Połączenie z ADLS Gen2      | 17 |
| <b>ZAŁĄCZNIK B</b> | Ściągawka SQL               | 18 |
| <b>ZAŁĄCZNIK C</b> | Dokumentacja                | 19 |

## MODUŁ 01

# Platforma i workspace

## Architektura lakehouse

Lakehouse łączy otwarte i tanie przechowywanie danych, typowe dla data lake, z niezawodnością i nadzorem charakterystycznymi dla hurtowni danych. Databricks buduje go z trzech warstw.

| Warstwa        | Technologia                               | Zastosowanie                                                           |
|----------------|-------------------------------------------|------------------------------------------------------------------------|
| Przechowywanie | Delta Lake w chmurowym magazynie obiektów | Transakcje ACID, wersjonowanie, kontrola schematu                      |
| Obliczenia     | Klasy Spark, SQL warehouse, serverless    | Przetwarzanie wsadowe i strumieniowe, uczenie maszynowe, analityka SQL |
| Nadzór         | Unity Catalog                             | Kontrola dostępu, lineage, wyszukiwanie danych                         |

## Zasoby obliczeniowe

| Typ                 | Typowe zastosowanie                               | Działanie                                                                 |
|---------------------|---------------------------------------------------|---------------------------------------------------------------------------|
| Klaster all-purpose | Interaktywne notebooki i development              | Działa do zatrzymania; opłata za czas działania                           |
| Klaster job         | Zadania harmonogramowane i automatyczne           | Tworzony na czas uruchomienia i usuwany po jego zakończeniu               |
| SQL warehouse       | Edytor SQL i narzędzia BI przez JDBC lub ODBC     | Wstrzymuje się przy braku ruchu; zoptymalizowany pod równoległe zapytania |
| Serverless          | Notebooki, zadania i SQL bez zarządzania klastrem | Startuje w kilka sekund; opłata za sekundy użycia                         |

Przy nowych pracach serverless jest zwykle najlepszym wyborem domyślnym: startuje w kilka sekund, nie wymaga konfiguracji i nie generuje kosztów w czasie bezczynności.

## Ścieżki do plików: wolumeny zamiast DBFS

|                  | DBFS (legacy)                              | Wolumen Unity Catalog                     |
|------------------|--------------------------------------------|-------------------------------------------|
| Ścieżka          | dbfs:/..., dbfs:/mnt/...                   | /Volumes/<catalog>/<schema>/<volume>      |
| Nadzór           | Brak; dostęp ma każdy użytkownik workspace | Uprawnienia Unity Catalog, audyt, lineage |
| Nowe workspace'y | Niedostępne                                | Domyślnie                                 |

DBFS root i mounty są wycofywane. Konta Azure Databricks utworzone po 18 grudnia 2025 r. nie mają do nich dostępu, podobnie jak do Hive Metastore i klastrów no-isolation shared. Od 30 września 2026 r. każdy nowo tworzony workspace powstaje bez tych funkcji, niezależnie od wieku konta. W nowym kodzie używaj ścieżek wolumenów.

## Polecenia magiczne

| Polecenie              | Działanie                                         |
|------------------------|---------------------------------------------------|
| %sql                   | Wykonuje komórkę jako SQL                         |
| %python                | Wykonuje komórkę jako Python                      |
| %sh                    | Uruchamia polecenia powłoki na driverze           |
| %pip install <package> | Instaluje bibliotekę na czas sesji notebooka      |
| %run ./path/notebook   | Uruchamia inny notebook w bieżącym kontekście     |
| %fs ls /Volumes/...    | Wyświetla pliki w wolumenie                       |
| %skip                  | Pomija komórkę przy uruchamianiu całego notebooka |

## Genie Code, asystent w notebookach

Od marca 2026 r. asystent znany wcześniej jako Databricks Assistant nosi nazwę Genie Code. Korzysta z metadanych Unity Catalog, więc jego podpowiedzi odnoszą się do tabel i kolumn w Twoim katalogu.

| Czynność            | Sposób                                                |
|---------------------|-------------------------------------------------------|
| Polecenie w komórce | Cmd+I w macOS, Ctrl+I w Windows                       |
| Wyjaśnienie kodu    | /explain                                              |
| Naprawa błędu       | /fix albo <b>Diagnose error</b> przy komórce z błędem |
| Dokumentacja kodu   | /doc                                                  |
| Szybsza wersja kodu | /optimize                                             |
| Formatowanie        | /prettify                                             |

Tryb agenta (Agent mode), dostępny ogólnie od marca 2026 r., samodzielnie wykonuje wieloetapowe zadania z zakresu inżynierii danych, data science i tworzenia dashboardów. Polecenia /settings i /rename usunięto w maju 2026 r.

Przed uruchomieniem wygenerowanego kodu zawsze go przeczytaj, zwłaszcza jeśli zapisuje lub usuwa dane.

## Widgety

```
dbutils.widgets.text("env", "dev", "Environment")
dbutils.widgets.dropdown("format", "delta", ["delta", "parquet", "csv"])
env = dbutils.widgets.get("env")
dbutils.widgets.removeAll()
```

## MODUŁ 02

# Architektura Spark

## Driver i executory

| Element  | Rola                                                                                            |
|----------|-------------------------------------------------------------------------------------------------|
| Driver   | Wykonuje Twój kod, buduje plan wykonania, planuje zadania i zbiera wyniki. Jeden na klaster.    |
| Executor | Wykonuje zadania równolegle, każde na jednej partycji w pamięci lub na dysku. Kilka na klaster. |
| Partycja | Fragment danych przetwarzany przez jedno zadanie. Domyślny rozmiar docelowy to 128 MB.          |

## Transformacje i akcje

| Pojęcie               | Znaczenie                                           | Przykłady                                                                                |
|-----------------------|-----------------------------------------------------|------------------------------------------------------------------------------------------|
| Transformacja         | Leniwa: definiuje nowy DataFrame, nie obliczając go | <code>filter</code> , <code>select</code> , <code>join</code> , <code>groupBy</code>     |
| Akcja                 | Uruchamia obliczenia i zwraca lub zapisuje wynik    | <code>count()</code> , <code>show()</code> , <code>write</code> , <code>collect()</code> |
| Transformacja wąska   | Działa w obrębie partycji; dane nie są przenoszone  | <code>filter</code> , <code>select</code>                                                |
| Transformacja szeroka | Rozdziela dane między partycje (shuffle)            | <code>groupBy</code> , <code>join</code> , <code>distinct</code>                         |

## DAG, etapy i zadania

Spark zamienia ciąg transformacji w skierowany graf acykliczny (DAG). Graf jest dzielony na etapy (stages) przy każdym shuffle, a każdy etap wykonuje się jako jedno zadanie na partycję. Shuffle przesyła dane przez sieć między executorami i w większości zadań jest najkosztowniejszą operacją.

## Catalyst i adaptive query execution

| Mechanizm                      | Kiedy działa       | Co robi                                                                                                                                      |
|--------------------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Optymalizator Catalyst         | Podczas planowania | Przekształca plan logiczny w fizyczny: przenosi filtry bliżej źródła, pomija zbędne kolumny, upraszcza wyrażenia stałe                       |
| Adaptive query execution (AQE) | Podczas wykonania  | Zmienia plan na podstawie rzeczywistych statystyk shuffle: łączy małe partycje, zmienia strategię joinów, dzieli partycje z przekosem danych |

AQE jest domyślnie włączone w Databricks Runtime. DataFrame API i `spark.sql()` kompilują się do tego samego planu, więc wybierz zapis, który jest czytelniejszy.

## MODUŁ 03

# ELT i ingestia

## Odczyt plików

```
# CSV: an explicit schema avoids a second pass over large files
df = (spark.read.format("csv")
      .option("header", True)
      .option("sep", ",")
      .option("nullValue", "NULL")
      .schema(my_schema)
      .load("/Volumes/<catalog>/<schema>/<volume>/customers.csv"))

# JSON with records spread over several lines
df = spark.read.option("multiLine", True).json("/Volumes/.../orders/")

# Parquet and Delta carry their own schema
df = spark.read.parquet("/Volumes/.../products/")
df = spark.read.table("catalog.schema.table")
```

## Eksploracja DataFrame

| Metoda                                  | Wynik                                                                 |
|-----------------------------------------|-----------------------------------------------------------------------|
| <code>display(df)</code>                | Interaktywna tabela; wykresy i profil danych są dostępne w menu +     |
| <code>df.show(5, truncate=False)</code> | Podgląd w formie tekstu                                               |
| <code>df.printSchema()</code>           | Nazwy i typy kolumn                                                   |
| <code>df.describe()</code>              | Liczność, średnia, odchylenie standardowe, minimum i maksimum         |
| <code>df.summary()</code>               | To samo co <code>describe()</code> , a do tego percentyle 25, 50 i 75 |

## Najczęstsze transformacje

| Operacja                   | Składnia                                                   |
|----------------------------|------------------------------------------------------------|
| Wybór kolumn               | <code>df.select("col1", "col2")</code>                     |
| Dodanie lub zmiana kolumny | <code>df.withColumn("new", col("x") * 2)</code>            |
| Zmiana nazwy kolumny       | <code>df.withColumnRenamed("old", "new")</code>            |
| Filtrowanie wierszy        | <code>df.filter(col("age") &gt; 18)</code>                 |
| Wartość warunkowa          | <code>when(col("x") &gt; 0, "pos").otherwise("neg")</code> |
| Usunięcie kolumny          | <code>df.drop("col")</code>                                |
| Zmiana typu                | <code>col("id").cast("integer")</code>                     |
| Usunięcie duplikatów       | <code>df.dropDuplicates(["col1", "col2"])</code>           |
| Sortowanie                 | <code>df.orderBy(col("date").desc())</code>                |
| Obsługa wartości null      | <code>df.fillna({"col": 0}), df.dropna()</code>            |

## Agregacja

```
from pyspark.sql.functions import count, sum, avg, col

(df.groupBy("region")
 .agg(count("*").alias("orders"),
      sum("amount").alias("revenue"),
      avg("amount").alias("avg_order"))
 .filter(col("orders") > 100))      # equivalent of SQL HAVING
```

## Joiny

| Typ   | Zwracane wiersze                                                                |
|-------|---------------------------------------------------------------------------------|
| inner | Wiersze pasujące w obu DataFrame'ach                                            |
| left  | Wszystkie wiersze z lewej strony, z dopasowaniami z prawej lub wartościami null |
| right | Wszystkie wiersze z prawej strony, z dopasowaniami z lewej lub wartościami null |
| full  | Wszystkie wiersze z obu stron                                                   |
| cross | Wszystkie kombinacje wierszy                                                    |

```
orders.join(customers, orders.customer_id == customers.id, "left")
```



## Widoki tymczasowe

```
df.createOrReplaceTempView("orders_view")
spark.sql("SELECT region, SUM(amount) FROM orders_view GROUP BY region").show()
```

## Zagnieżdżony JSON

```
from pyspark.sql.functions import explode, get_json_object, col

df.withColumn("item", explode(col("items"))) # array to rows
df.withColumn("city", get_json_object(col("json_str"), "$.address.city"))
```

## MODUŁ 04

# Delta Lake

Delta Lake przechowuje dane w plikach Parquet wraz z dziennikiem transakcji w katalogu `_delta_log/`. Każda zmiana trafia do dziennika i to właśnie umożliwia transakcje ACID, time travel oraz kontrolę schematu.

## Gwarancje ACID

| Własność               | Znaczenie                                                                |
|------------------------|--------------------------------------------------------------------------|
| Atomowość (atomicity)  | Zapis kończy się w całości albo nie zostawia żadnego śladu               |
| Spójność (consistency) | Schemat i ograniczenia są sprawdzane przy każdym zapisie                 |
| Izolacja (isolation)   | Równoległe czytający i zapisujący nie widzą niedokończonych zmian innych |
| Trwałość (durability)  | Zatwierdzone dane przetrwają awarię                                      |

## Modyfikacja danych

```
INSERT INTO catalog.schema.orders VALUES (1001, 'C001', 99.99, current_date());

UPDATE catalog.schema.orders SET status = 'shipped' WHERE order_id = 1001;

DELETE FROM catalog.schema.orders WHERE status = 'cancelled';

-- Upsert: update matches, insert new rows, delete rows missing from the source
MERGE INTO target t
USING source s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET t.val = s.val
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE THEN DELETE;
```

## Time travel

```
DESCRIBE HISTORY my_table;

SELECT * FROM my_table VERSION AS OF 5;
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';

RESTORE TABLE my_table TO VERSION AS OF 5;
```

```
df = spark.read.option("versionAsOf", 5).table("my_table")
```

## VACUUM

```
VACUUM my_table DRY RUN;           -- list the files that would be removed
VACUUM my_table;                   -- remove files older than 7 days (the default)
VACUUM my_table RETAIN 168 HOURS;  -- the same, with the retention stated explicitly
```

Plików usuniętych przez VACUUM nie da się odzyskać, a wersje tabeli, które od nich zależały, przestają być dostępne przez time travel. Na tabelach czytanych równolegle utrzymuj retencję co najmniej 7 dni.

## OPTIMIZE i Z-ordering

```
OPTIMIZE my_table;                  -- compact small files
OPTIMIZE my_table ZORDER BY (order_date); -- compact and co-locate by column
```

Z-ordering umieszcza wiersze o podobnych wartościach w tych samych plikach, więc filtry na tych kolumnach odczytują mniej plików. Najlepiej sprawdza się na kolumnach często filtrowanych i mających wiele różnych wartości.

## Tabele zarządzane i zewnętrzne

|                    | Tabela zarządzana                      | Tabela zewnętrzna                                   |
|--------------------|----------------------------------------|-----------------------------------------------------|
| Lokalizacja danych | Magazyn zarządzany przez Unity Catalog | Wskazana ścieżka w chmurze (abfss://, s3://, gs://) |
| DROP TABLE         | Usuwa metadane i pliki danych          | Usuwa tylko metadane; pliki pozostają               |
| Kiedy stosować     | Domyślnie                              | Gdy dane muszą pozostać w istniejącej lokalizacji   |

## MODUŁ 05

# Orkestracja w Lakeflow Jobs

## Pojęcia

| Termin      | Znaczenie                                                                                   |
|-------------|---------------------------------------------------------------------------------------------|
| Job         | Jednostka uruchamiana według harmonogramu lub na żądanie, złożona z jednego lub kilku zadań |
| Task        | Pojedynczy krok: notebook, skrypt Python, zapytanie SQL, pipeline i inne                    |
| Zależność   | Kolejność wykonywania zadań; zadania joba tworzą DAG                                        |
| Repair run  | Ponowne uruchomienie tylko zadań zakończonych błędem i zadań od nich zależnych              |
| Klaster job | Zasoby tworzone na czas uruchomienia i usuwane po jego zakończeniu                          |

## Wyzwalacze

| Wyzwalacz    | Uruchamia job                                            |
|--------------|----------------------------------------------------------|
| Scheduled    | Według harmonogramu cron                                 |
| File arrival | Gdy we wskazanej lokalizacji pojawią się nowe pliki      |
| Continuous   | Ponownie, zaraz po zakończeniu poprzedniego uruchomienia |
| Ręcznie      | Na żądanie, z interfejsu lub przez API                   |

## Wyrażenia cron

Databricks używa formatu Quartz, który zaczyna się od pola sekund.

| sek. | min. | godz. | dzień | mies. | mies. | dzień   | tyg. |                                    |
|------|------|-------|-------|-------|-------|---------|------|------------------------------------|
| 0    | 30   | 2     | *     | *     | *     | ?       |      | codziennie o 02:30                 |
| 0    | 0    | 8     | ?     | *     | *     | MON-FRI |      | w dni robocze o 08:00              |
| 0    | 0    | 0     | 1     | *     | *     | ?       |      | pierwszego dnia miesiąca o północy |

## Przekazywanie wartości między zadaniami

```
# Read a job parameter
dbutils.widgets.text("source_table", "samples.nyctaxi.trips")
source_table = dbutils.widgets.get("source_table")

# Return a value to downstream tasks
import json
dbutils.notebook.exit(json.dumps({"status": "SUCCESS", "rows": row_count}))

# Read the value returned by an upstream task
result = dbutils.jobs.taskValues.get(taskKey="validate", key="returnValue", default="{")
```

## Lakeflow Pipelines

Lakeflow Pipelines, dawniej Delta Live Tables, pozwala deklaratywnie definiować tabele, a zależnościami między nimi i ich odświeżaniem zarządza platforma.

| Obiekt            | Działanie                                                                         |
|-------------------|-----------------------------------------------------------------------------------|
| Streaming table   | Pobiera dane przyrostowo, przetwarzając tylko dane nowe od ostatniej aktualizacji |
| Materialized view | Przechowuje wynik zapytania i odświeża go przy każdej aktualizacji pipeline'u     |
| Widok             | Nazwane zapytanie używane wewnątrz pipeline'u; nie jest zapisywane                |

```
CREATE OR REFRESH STREAMING TABLE bronze_orders
AS SELECT * FROM STREAM(LIVE.raw_landing);

CREATE OR REFRESH MATERIALIZED VIEW silver_orders
AS SELECT order_id, customer_id, amount
   FROM LIVE.bronze_orders
  WHERE amount > 0;
```

## Monitorowanie nieudanych uruchomień

```
SELECT *
FROM system.lakeflow.job_run_timeline
WHERE result_state = 'FAILED'
   AND period_start_time >= current_timestamp() - INTERVAL 7 DAYS;
```

Odczyt tabel systemowych wymaga dostępu do katalogu system.

## MODUŁ 06

# Unity Catalog

## Hierarchia obiektów

```
Metastore          jeden na region, wspólny dla workspace'ów w tym regionie
└─ Catalog          na przykład dev, prod, sandbox
   └─ Schema         logiczna grupa obiektów, odpowiednik bazy danych
      └─ Tabela, widok, wolumen, funkcja
```

Odwołuj się do obiektów pełną, trzyczłonową nazwą `catalog.schema.table`, na przykład `main.retail.orders`.

## Nadawanie uprawnień

Uprawnienia trzeba nadać na każdym poziomie, zaczynając od katalogu.

```
GRANT USE CATALOG ON CATALOG main TO `retailhub_analysts`;
GRANT USE SCHEMA ON SCHEMA main.retail TO `retailhub_analysts`;
GRANT SELECT ON TABLE main.retail.orders TO `retailhub_analysts`;

SHOW GRANTS ON TABLE main.retail.orders;

REVOKE SELECT ON TABLE main.retail.orders FROM `retailhub_analysts`;
```

| Uprawnienie    | Nadawane na                                           | Pozwala na                                       |
|----------------|-------------------------------------------------------|--------------------------------------------------|
| USE CATALOG    | Katalog                                               | Otwarcie katalogu; wymagane jako pierwsze        |
| USE SCHEMA     | Schemat                                               | Otwarcie schematu                                |
| SELECT         | Tabela, widok lub schemat (dla wszystkich jego tabel) | Odczyt danych                                    |
| MODIFY         | Tabela                                                | Wstawianie, aktualizację i usuwanie danych       |
| CREATE TABLE   | Schemat                                               | Tworzenie tabel                                  |
| MANAGE         | Dowolny obiekt                                        | Nadawanie i odbieranie uprawnień do tego obiektu |
| ALL PRIVILEGES | Dowolny obiekt                                        | Większość uprawnień, ale nie <b>MANAGE</b>       |

Uprawnienia do obiektu może nadawać tylko jego właściciel, administrator metastore albo posiadacz **MANAGE**. Samo **ALL PRIVILEGES** nie wystarcza.

## Grupy

Nadawaj uprawnienia grupom, a nie pojedynczym użytkownikom. Unity Catalog akceptuje wyłącznie grupy kontowe.

|                                         | Grupa kontowa                                                             | Grupa lokalna workspace                                              |
|-----------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------|
| Tworzona w                              | Konsoli konta albo w ustawieniach workspace, w sekcji Identity and access | Workspace Groups API albo poleceniem SQL <code>CREATE GROUP</code>   |
| Może otrzymać uprawnienia Unity Catalog | Tak                                                                       | Nie; nadanie kończy się błędem <code>PRINCIPAL_DOES_NOT_EXIST</code> |

Polecenie SQL `CREATE GROUP` wykonuje się bez błędu, ale tworzy grupę lokalną workspace. Grupy twórz w konsoli konta albo w ustawieniach workspace.

## Zewnętrzne magazyny danych

| Obiekt             | Zastosowanie                                                                                                |
|--------------------|-------------------------------------------------------------------------------------------------------------|
| Storage credential | Przechowuje tożsamość w chmurze, na przykład Azure Access Connector, używaną do dostępu do magazynu         |
| External location  | Łączy ścieżkę w magazynie z credentialem i kontroluje <code>READ FILES</code> oraz <code>WRITE FILES</code> |
| Tabela zewnętrzna  | Tabela, której pliki są przechowywane w external location                                                   |

## MODUŁ 07

# Dashboardy AI/BI i Genie

## Dashboardy AI/BI

Dashboardy prezentują dane z Unity Catalog jako wykresy, liczniki i tabele, z wyborem dat, filtrami rozwijanymi i filtrowaniem krzyżowym między widżetami. Opublikowany dashboard ma link do udostępniania, nie wymaga od odbiorców uruchamiania zapytań, może być wysyłany e-mailem według harmonogramu i osadzany w innych aplikacjach.

## Przestrzeń Genie

Przestrzeń Genie odpowiada na pytania zadawane językiem naturalnym, generując i wykonując SQL na skonfigurowanych dla niej tabelach. Pamięta kontekst rozmowy, więc działają pytania uzupełniające, a odpowiedzi można zapisać jako widżety dashboardu. Genie dobrze sprawdza się przy eksploracyjnych, samoobsługowych pytaniach; gorzej radzi sobie z wieloetapową logiką i raportami o ściśle określonym formacie.

## Wybór narzędzia

| Potrzeba                                                                                 | Narzędzie        |
|------------------------------------------------------------------------------------------|------------------|
| Zatwierdzony dashboard KPI odświeżany codziennie                                         | Dashboard AI/BI  |
| Doraźne pytanie, np. o dziesięć najlepiej sprzedających się produktów w zeszłym miesiącu | Przestrzeń Genie |
| Eksploracja danych przed zbudowaniem dashboardu                                          | Edytor SQL       |



## ZAŁĄCZNIK A

# Połączenie z ADLS Gen2

| Krok | Gdzie                                             | Czynność                                                                                                             |
|------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| 1    | Azure Portal, zarządzana grupa zasobów Databricks | Skopiuj identyfikator zasobu Access Connectora ( <code>Microsoft.Databricks/accessConnectors/&lt;name&gt;</code> )   |
| 2    | Konto magazynu, Access control (IAM)              | Przypisz rolę <b>Storage Blob Data Contributor</b> tożsamości zarządzanej Access Connectora                          |
| 3    | Catalog Explorer, External data, Credentials      | Utwórz storage credential z identyfikatorem zasobu Access Connectora                                                 |
| 4    | Catalog Explorer, External locations              | Utwórz external location z adresu <code>abfss://</code> i credentialu                                                |
| 5    | External location, Permissions                    | Nadaj <code>READ FILES</code> , <code>WRITE FILES</code> lub <code>CREATE EXTERNAL TABLE</code> zgodnie z potrzebami |

```
SHOW EXTERNAL LOCATIONS;  
LIST 'abfss://container@account.dfs.core.windows.net/';
```

Dla produkcyjnych źródeł tylko do odczytu przypisz rolę Storage Blob Data Reader zamiast Contributor i nadawaj pojedyncze uprawnienia Unity Catalog zamiast `ALL PRIVILEGES`.

## ZAŁĄCZNIK B

# Ściągawka SQL

---

## Delta Lake

```
DESCRIBE DETAIL my_table;  
DESCRIBE HISTORY my_table;  
OPTIMIZE my_table ZORDER BY (col);  
VACUUM my_table RETAIN 168 HOURS;  
RESTORE TABLE my_table TO VERSION AS OF 3;  
SELECT * FROM my_table VERSION AS OF 2;  
SELECT * FROM my_table TIMESTAMP AS OF '2026-01-15 08:00:00';
```

## Unity Catalog

```
SHOW CATALOGS;  
SHOW SCHEMAS IN my_catalog;  
SHOW TABLES IN my_catalog.my_schema;  
DESCRIBE TABLE EXTENDED my_catalog.my_schema.my_table;  
SHOW GRANTS ON TABLE my_catalog.my_schema.my_table;
```

## ZALĄCZNIK C

# Dokumentacja

| Temat                                    | Adres                                                                                                                                                                                     |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Dokumentacja Databricks                  | <a href="https://docs.databricks.com">docs.databricks.com</a>                                                                                                                             |
| Delta Lake                               | <a href="https://docs.delta.io">docs.delta.io</a>                                                                                                                                         |
| Apache Spark                             | <a href="https://spark.apache.org/docs/latest">spark.apache.org/docs/latest</a>                                                                                                           |
| Lakeflow Jobs                            | <a href="https://docs.databricks.com/en/workflows/index.html">docs.databricks.com/en/workflows/index.html</a>                                                                             |
| Lakeflow Pipelines                       | <a href="https://docs.databricks.com/en/dlt/index.html">docs.databricks.com/en/dlt/index.html</a>                                                                                         |
| Unity Catalog                            | <a href="https://docs.databricks.com/en/data-governance/unity-catalog/index.html">docs.databricks.com/en/data-governance/unity-catalog/index.html</a>                                     |
| Uprawnienia w Unity Catalog              | <a href="https://docs.databricks.com/en/data-governance/unity-catalog/manage-privileges/index.html">docs.databricks.com/en/data-governance/unity-catalog/manage-privileges/index.html</a> |
| Storage credentials i external locations | <a href="https://docs.databricks.com/en/connect/unity-catalog/external-locations.html">docs.databricks.com/en/connect/unity-catalog/external-locations.html</a>                           |
| Serverless                               | <a href="https://docs.databricks.com/en/compute/serverless.html">docs.databricks.com/en/compute/serverless.html</a>                                                                       |
| Adaptive query execution                 | <a href="https://docs.databricks.com/en/optimizations/aqe.html">docs.databricks.com/en/optimizations/aqe.html</a>                                                                         |
| OPTIMIZE i Z-ordering                    | <a href="https://docs.databricks.com/en/delta/optimize.html">docs.databricks.com/en/delta/optimize.html</a>                                                                               |
| Dashboardy AI/BI                         | <a href="https://docs.databricks.com/en/dashboards/index.html">docs.databricks.com/en/dashboards/index.html</a>                                                                           |
| AI/BI Genie                              | <a href="https://docs.databricks.com/en/genie/index.html">docs.databricks.com/en/genie/index.html</a>                                                                                     |
| Genie Code                               | <a href="https://docs.databricks.com/en/genie-code/">docs.databricks.com/en/genie-code/</a>                                                                                               |
| Wycofanie DBFS i innych funkcji legacy   | <a href="https://docs.databricks.com/en/admin/account-settings/legacy-features.html">docs.databricks.com/en/admin/account-settings/legacy-features.html</a>                               |